Tabelas de dispersão

Quando a chave diz onde ela mora

Encontro 10 | 1º de outubro

Estruturas de Dados Avançadas

Onde chegamos

lista	buscar custa n
árvore balanceada	buscar custa $\log n$

Onde chegamos

lista	buscar custa n
árvore balanceada	buscar custa $\log n$

Com um milhão de chaves, a árvore faz 20 comparações.

Onde chegamos

lista	buscar custa n
árvore balanceada	buscar custa $\log n$

Com um milhão de chaves, a árvore faz 20 comparações.

Dá para fazer melhor que 20?

Dá: uma

E não é uma a mais por chave adicionada.

Dá: uma

E não é uma a mais por chave adicionada.

É uma, com mil ou com um bilhão.

Dá: uma

E não é uma a mais por chave adicionada.

É uma, com mil ou com um bilhão.

Falta descobrir como, e o que se paga por isso.

1

Como funciona

Parar de procurar

Na árvore, você **procura** a chave descendo e comparando.

Parar de procurar

Na árvore, você **procura** a chave descendo e comparando.

E se a chave dissesse onde ela está?

Parar de procurar

Na árvore, você **procura** a chave descendo e comparando.

E se a chave dissesse onde ela está?

Aí não há busca. Calcula-se o endereço e vai direto.

A conta que decide a gaveta



A conta que decide a gaveta



Transforme a chave num número, e tire o resto da divisão pelo tamanho da tabela.

Três chaves, três contas

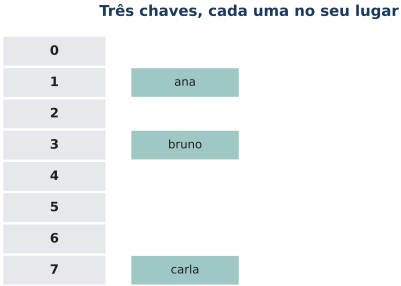
chave	vira o número	resto por 8
ana	7 431 009	1
bruno	2 885 443	3
carla	9 120 767	7

Três chaves, três contas

chave	vira o número	resto por 8
`ana`	7 431 009	1
`bruno`	2 885 443	3
`carla`	9 120 767	7

Essa conta chama-se **função de dispersão**.

E elas vão direto

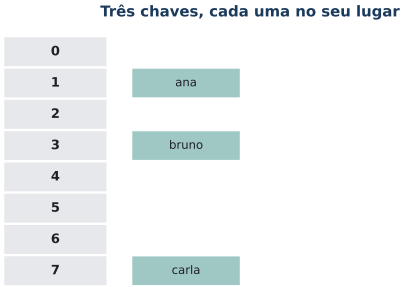


E elas vão direto



Sem procurar. Sem comparar. A conta disse o lugar.

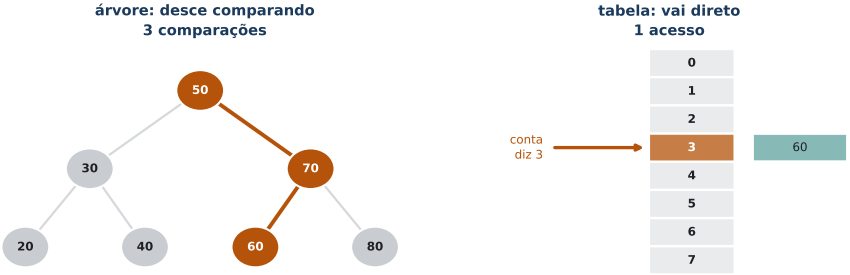
E elas vão direto



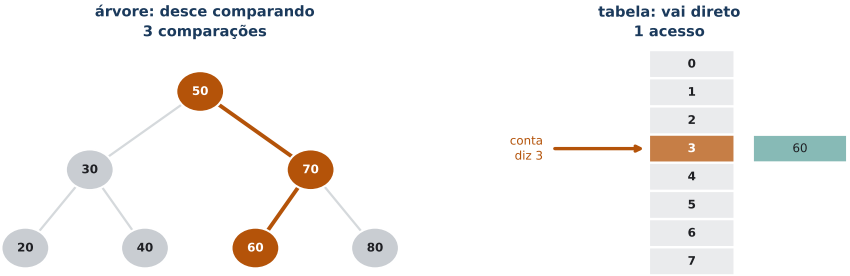
Sem procurar. Sem comparar. A conta disse o lugar.

Para buscar, faz-se a mesma conta e olha-se a gaveta.

A diferença, lado a lado



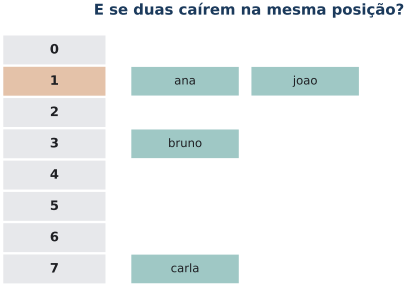
A diferença, lado a lado



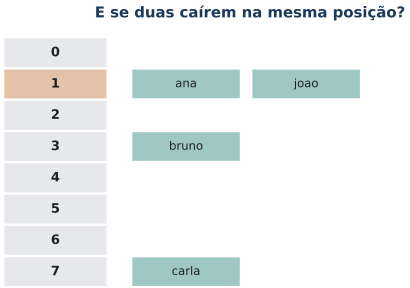
À esquerda, três perguntas até chegar. À direita, uma conta e pronto.

2
O problema

Duas chaves, a mesma gaveta

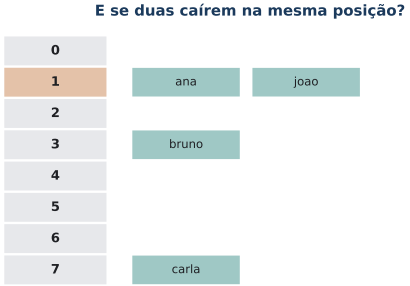


Duas chaves, a mesma gaveta



joao também caiu na posição 1. Isso se chama **colisão**.

Duas chaves, a mesma gaveta



joao também caiu na posição 1. Isso se chama **colisão**.

E é inevitável: há mais chaves possíveis que gavetas.

A solução


Cada posição guarda uma lista
0
1
ana
joao
2
iris
luis
3
bruno
elisa
gabi
4
5
davi
6
7
carla

A solução



Cada gaveta guarda uma **lista**. Quem colide entra na fila.

A solução



Cada gaveta guarda uma **lista**. Quem colide entra na fila.

Buscar `gabi`: vai direto à gaveta 3, e compara os três de lá.

O custo depende de quantas gavetas há

Com **mil gavetas**, guardando quantidades diferentes de chaves:

chaves guardadas	a gaveta mais cheia tem	comparações por busca
100	1 chave	**1,0**
1.000	4 chaves	**1,4**
10.000	14 chaves	**5,6**

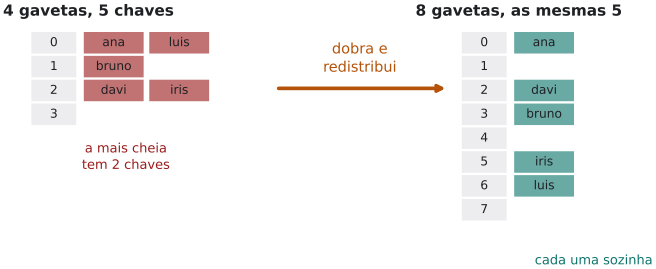
O custo depende de quantas gavetas há

Com **mil gavetas**, guardando quantidades diferentes de chaves:

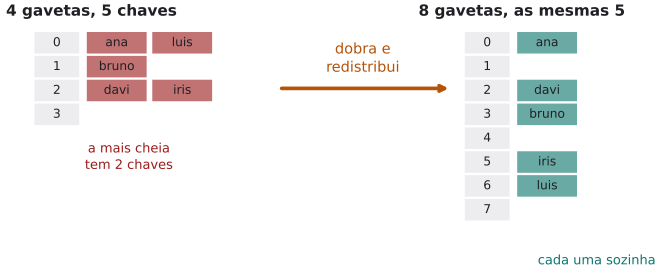
chaves guardadas	a gaveta mais cheia tem	comparações por busca
100	1 chave	**1,0**
1.000	4 chaves	**1,4**
10.000	14 chaves	**5,6**

Enquanto houver gaveta sobrando, as listas ficam curtas
e a busca custa quase 1.

Por isso a tabela cresce sozinha

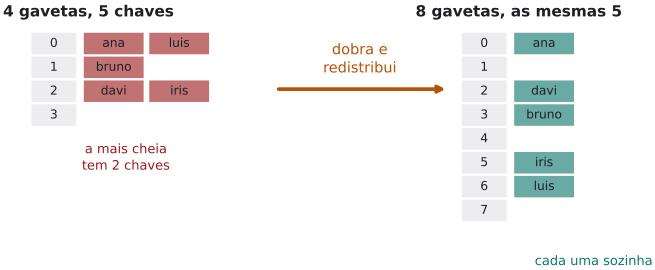


Por isso a tabela cresce sozinha



Com mais gavetas, as mesmas chaves se espalham e as listas somem.

Por isso a tabela cresce sozinha



Com mais gavetas, as mesmas chaves se espalham e as listas somem.

Redistribuir custa caro, mas acontece poucas vezes.
E o Python faz sozinho: você não precisa pedir.

3
Contra a árvore

Medido, com 100 mil chaves

	árvore	tabela	razão
construir	1,123 s	**0,023 s**	**48x**
buscar 50 mil	0,064 s	**0,011 s**	**6x**
comparações por busca	15,9	**1**	

Medido, com 100 mil chaves

	árvore	tabela	razão
construir	1,123 s	**0,023 s**	**48x**
buscar 50 mil	0,064 s	**0,011 s**	**6x**
comparações por busca	15,9	**1**	

A tabela ganha em tudo. Ou quase.

A pergunta que vira o jogo

“Me dê todas as chaves entre 1000 e 2000.”

A pergunta que vira o jogo

“Me dê todas as chaves entre 1000 e 2000.”

Na árvore: desce até o 1000 e percorre em ordem até passar do 2000.

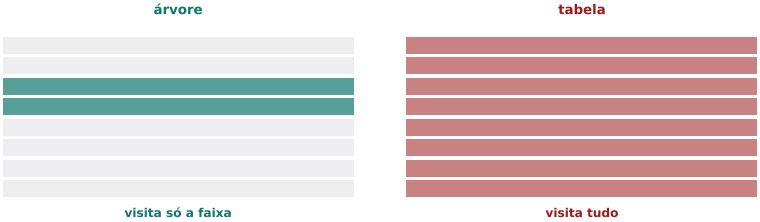
A pergunta que vira o jogo

"Me dê todas as chaves entre 1000 e 2000."

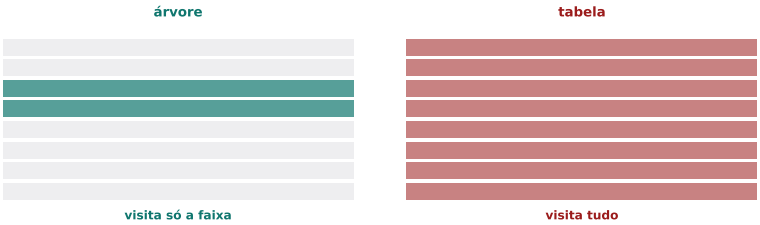
Na árvore: desce até o 1000 e percorre em ordem até passar do 2000.

Na tabela: não há ordem nenhuma.
É preciso olhar **todas** as gavetas.

O que cada uma precisa olhar



O que cada uma precisa olhar



A árvore desce até o começo da faixa e para quando passa do fim.
A tabela não tem como saber onde a faixa está.

Medido também

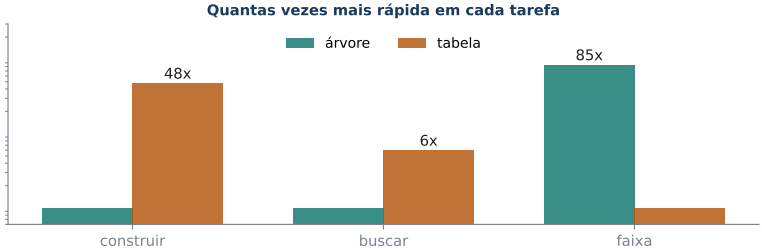
árvore	**0,07 ms** (99 chaves visitadas)
tabela	**6,14 ms** (100 mil visitadas)
razão	**85x** a favor da árvore

Medido também

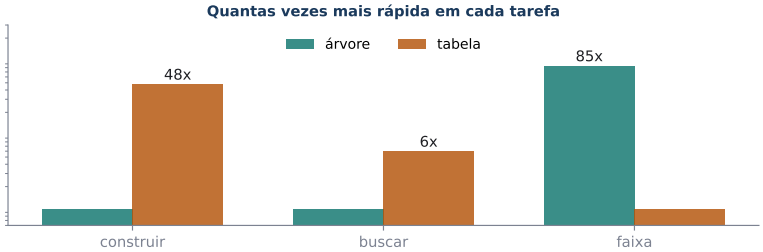
árvore	**0,07 ms** (99 chaves visitadas)
tabela	**6,14 ms** (100 mil visitadas)
razão	**85x** a favor da árvore

E a diferença cresce com o tamanho dos dados.

O placar


Quantas vezes mais rápida em cada tarefa
árvore
tabela
48x
6x
85x
construir
buscar
faixa

O placar



Cada uma ganha em alguma coisa, e por muito.

O quadro completo

	árvore	**tabela**
buscar uma chave	$\log n$	1
inserir	$\log n$	1
listar em ordem	sai pronto	precisa ordenar
chaves numa faixa	só as da faixa	todas

O quadro completo

	árvore	**tabela**
buscar uma chave	$\log n$	1
inserir	$\log n$	1
listar em ordem	sai pronto	precisa ordenar
chaves numa faixa	só as da faixa	todas

A tabela é mais rápida. A árvore sabe a ordem.

Como escolher

só busca por chave exata	tabela de dispersão
precisa de ordem ou faixa	árvore balanceada

Como escolher

só busca por chave exata	tabela de dispersão
precisa de ordem ou faixa	árvore balanceada

Cadastro consultado por CPF: tabela.
Consulta com `ORDER BY` ou `BETWEEN`: árvore.

Como escolher

só busca por chave exata	tabela de dispersão
precisa de ordem ou faixa	árvore balanceada

Cadastro consultado por CPF: tabela.
Consulta com `ORDER BY` ou `BETWEEN`: árvore.

É por isso que bancos de dados usam árvore nos índices,
mesmo a tabela sendo mais rápida para buscar um registro.

O que fica

- Na tabela, a chave calcula o endereço em vez de ser procurada.

O que fica

- Na tabela, a chave calcula o endereço em vez de ser procurada.
- Buscar custa 1, com mil ou com um bilhão de chaves.

O que fica

- Na tabela, a chave calcula o endereço em vez de ser procurada.
- Buscar custa 1, com mil ou com um bilhão de chaves.
- Colisão é inevitável, e se resolve com uma lista por gaveta.

O que fica

- Na tabela, a chave calcula o endereço em vez de ser procurada.
- Buscar custa 1, com mil ou com um bilhão de chaves.
- Colisão é inevitável, e se resolve com uma lista por gaveta.
- A tabela ganha em buscar e inserir: 48 vezes para construir.

O que fica

- Na tabela, a chave calcula o endereço em vez de ser procurada.
- Buscar custa 1, com mil ou com um bilhão de chaves.
- Colisão é inevitável, e se resolve com uma lista por gaveta.
- A tabela ganha em buscar e inserir: 48 vezes para construir.
- E perde em qualquer coisa que envolva ordem: 85 vezes.

O que fica

- Na tabela, a chave calcula o endereço em vez de ser procurada.
- Buscar custa 1, com mil ou com um bilhão de chaves.
- Colisão é inevitável, e se resolve com uma lista por gaveta.
- A tabela ganha em buscar e inserir: 48 vezes para construir.
- E perde em qualquer coisa que envolva ordem: 85 vezes.

Não existe estrutura melhor.
Existe a adequada à pergunta que você faz.